ThePnLab

Plateforme de Paper Trading Algorithmique

Rapport d'Architecture Technique & Méthodologie

thepnlab.com

github.com/Hamza-67/ThePnLab

Hamza Houir Alami

ENSAM Lille (Arts et Métiers ParisTech)

2025–2026 · en cours

Double Architecture	**Gemini 2.0** Flash LLM	**36+** Actifs	**ML** Learning Layer	**37** Utilisateurs

Simulation uniquement — aucun capital réel impliqué.

Code source : thepnlab.com

Table des matières

1 Résumé exécutif **2**

2 Architecture système **2**
2.1 Double application . . . 2
2.2 Stack technologique complet . . . 3
2.3 Structure du dépôt . . . 3

3 Agent pédagogique — `tradinglab/agent.py` **3**
3.1 Logique de décision . . . 4
3.2 Injection d'erreurs délibérées . . . 4
3.3 Paramètres de réalisme . . . 5

4 Screener de production — `bot_screener.py` **5**
4.1 Univers d'actifs (36 tickers, 9 catégories) . . . 5
4.2 Architecture parallèle . . . 5
4.3 Indicateurs techniques calculés . . . 6
4.4 Fonction de score composite . . . 6

5 Moteur de décision LLM — `bot_brain.py` **7**
5.1 Cascade de modèles . . . 7
5.2 Couche d'apprentissage ML . . . 7
5.3 Prompt engineering — Cinq blocs de contexte . . . 8
5.4 Contraintes imposées au LLM . . . 8
5.5 Format de sortie JSON structuré . . . 9

6 Actualités & Sentiment macro — `news.py` **9**
6.1 Filtrage des sources . . . 10
6.2 Scoring de pertinence macro . . . 10
6.3 Score de risque → multiplicateur de position . . . 10

7 Gestion du risque **10**
7.1 Pipeline de dimensionnement en 4 couches . . . 10
7.2 Stratégie de sortie . . . 11

8 Lien avec la Recherche Opérationnelle **11**
8.1 Prise de décision séquentielle — Formalisation MDP . . . 11
8.2 Aide multicritère à la décision (AMCD) . . . 11
8.3 Contrôle stochastique & changement de régime . . . 12

9 Résultats & Validation **12**
9.1 Performance live — 30 derniers jours . . . 12

10 Limitations & Feuille de route **12**
10.1 Limitations actuelles . . . 12
10.2 Feuille de route recherche . . . 13

11 Références **13**

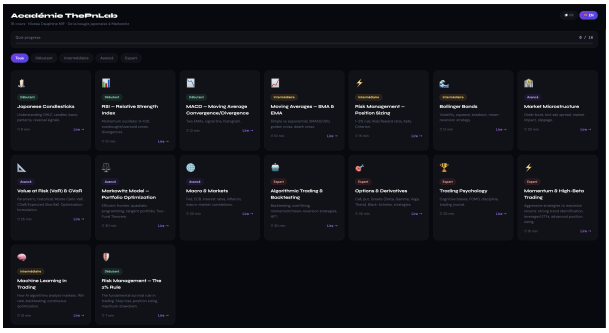
1. Résumé exécutif

ThePnLab est une plateforme de simulation de trading gratuite, conçue pour étudier la prise de décision algorithmique sur les marchés financiers sans capital réel. Elle s'adresse aux étudiants curieux de finance et aux futurs quants qui souhaitent développer une intuition concrète pour les stratégies quantitatives.

La plateforme repose sur une **double architecture** : une application Streamlit pédagogique où les utilisateurs observent (et apprennent de) un agent à règles simples avec des erreurs délibérées, et un backend FastAPI de production qui pilote un bot autonome guidé par LLM, s'exécutant quotidiennement à l'ouverture des marchés.

Chiffres clés — avril 2026 :
- **37 utilisateurs inscrits** — croissance 100% organique, aucune campagne marketing
- **36+ actifs** screenés quotidiennement dans 9 catégories (US tech, ETF, crypto, CAC40)
- **Cascade de 5 modèles** : Gemini 2.5-flash → 2.0-flash → 1.5-flash → 1.5-pro → Groq Llama-3.3-70B
- Capital de départ par utilisateur : 10 000 $ (simulé)
- Réalisme d'exécution : frais 0,1% par trade + slippage 5 bps

(a) *Vue portfolio — P&L en temps réel par utilisateur*



(b) *Académie — modules de cours structurés*

Figure 1 – *ThePnLab — captures de la plateforme en production (thepnlab.com)*

2. Architecture système

2.1. Double application

ThePnLab se déploie en deux applications distinctes aux rôles complémentaires :

	App Streamlit (Pédagogique)	Backend FastAPI (Production)
Rôle	Simulation & apprentissage	Bot de trading autonome IA
Moteur décision	Agent à règles (`agent.py`)	Cerveau LLM (`bot_brain.py`)
Planificateur	À la demande (déclenchement user)	APScheduler à l'ouverture
Base de données	SQLite (`tradinglab.db`)	PostgreSQL (Railway/Supabase)
Particularité	Injection d'erreurs 10%	Couche ML d'apprentissage

2.2. Stack technologique complet

Couche	Technologie
Frontend pédagogique	Streamlit + Plotly (Python)
Backend de production	FastAPI (Python 3.11)
Données de marché	yfinance — OHLCV fenêtre 6 mois, intervalle 1j
Moteur LLM	Google Gemini 2.0 Flash + Groq Llama-3.3-70B (secours)
Actualités / macro	NewsAPI — 50+ mots-clés macro, scoring de risque heuristique
BDD (développement)	SQLite + SQLAlchemy ORM
BDD (production)	PostgreSQL via Railway / Supabase
Screening parallèle	`ThreadPoolExecutor` (12 workers, timeout 30s)
Hébergement	thepnlab.com (en production)

2.3. Structure du dépôt

```
ThePnLab-main/
|-- app.py                      # Point d'entree Streamlit
|-- tradinglab/                 # Module de simulation pedagogique
|   |-- agent.py                # Agent a regles + injection d'erreurs
|   |-- indicators.py           # RSI, SMA50, BB, MACD (librairie ta)
|   |-- trading.py              # execute_buy/sell avec slippage + frais
|   |-- news.py                 # NewsAPI + scoring risque macro
|   |-- ai_gemini.py            # Coach Gemini (cascade de modeles)
|   |-- config.py               # STARTING_CASH, FEE_RATE, SLIPPAGE_BPS
|   `-- market.py               # Telechargement OHLCV via yfinance
`-- backend/app/
    |-- main.py                 # Factory FastAPI
    `-- routers/
        |-- bot_screener.py     # Screener parallele 36 actifs
        |-- bot_brain.py        # Moteur de decision LLM (Gemini + Groq)
        |-- bot_engine.py       # Execution des trades + gestion portfolio
        |-- coach.py            # Coach IA pedagogique
        `-- portfolio.py        # Etat portfolio & endpoints PnL
```

Listing 1 – *Arborescence du projet — ThePnLab-main/*

3. Agent pédagogique — `tradinglab/agent.py`

L'application Streamlit expose un agent de trading pédagogique dont la logique est volontairement simple et entièrement transparente. Chaque règle de décision est lisible directement dans l'interface.

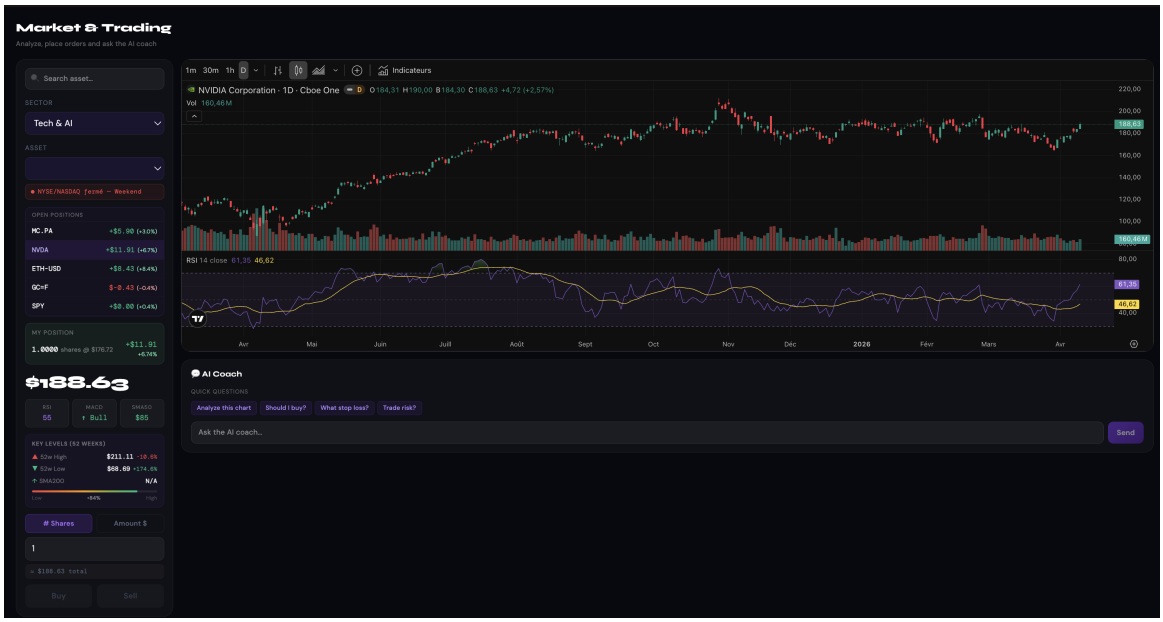


Figure 2 – *Vue Marché & Trades — les décisions de l'agent sont affichées avec leur justification complète en temps réel*

3.1. Logique de décision

L'agent utilise quatre signaux : RSI(14), MACD vs ligne de signal, prix vs SMA(50), et position en cours.

```python
def decide(rsi, macd, macd_signal, price, sma50,
           cash, have_qty, error_rate=0.10) -> Decision:

    trend_up    = price > sma50          # Filtre tendance SMA50
    over_sold   = rsi < 30               # Zone survendu RSI
    over_bought = rsi > 70               # Zone surachet RSI
    macd_bull   = macd > macd_signal     # Croisement MACD

    if over_sold and macd_bull and cash > price:
        side = "BUY"     # RSI bas + MACD haussier -> entree progressive
    elif over_bought and have_qty > 0:
        side = "SELL"    # Prise de profits / reduction du risque
    elif trend_up and macd_bull and cash > price:
        side = "BUY"     # Tendance > SMA50 + MACD: petit renfort
    elif (not trend_up) and have_qty > 0 and (not macd_bull):
        side = "SELL"    # Sous SMA50 + MACD en baisse: allegement
```

Listing 2 – *Fonction de décision principale — agent.py (verbatim)*

3.2. Injection d'erreurs délibérées

Choix pédagogique central : l'agent commet intentionnellement des erreurs avec une probabilité $\varepsilon = 0{,}10$ (10%). Cela enseigne aux utilisateurs qu'une stratégie structurellement correcte produit quand même des pertes, et que la *discipline d'exécution* compte autant que la qualité du signal.

```python
if random.random() < error_rate:          # 10% de chances d'erreur
    error_mode = "mistake"
    if side == "BUY":
        side = "HOLD"                     # hesitation / entree manquee
        rationale = "Erreur simulee: hesitation."
    elif side == "SELL":
        side = "HOLD"                     # conserve malgre le signal de vente
        rationale = "Erreur simulee: je garde malgre le signal."
```

```
    else:
        # trade impulsif aleatoire sur signal neutre
        side = "SELL" if (have_qty > 0 and random.random() < 0.5) else "BUY"
```

Listing 3 – *Mécanisme d'injection d'erreurs — agent.py*

3.3. Paramètres de réalisme

Paramètre	Valeur	Clé de config
Capital de départ	10 000 $	`STARTING_CASH = 10000`
Frais par trade	0,1%	`FEE_RATE = 0.001`
Slippage d'exécution	5 bps (0,05%)	`SLIPPAGE_BPS = 5`
Taille max position	15% du cash	`MAX_POSITION_PCT = 0.15`
Appels IA par jour	12 (plafond coût)	`AI_CALLS_PER_DAY = 12`

4. Screener de production — `bot_screener.py`

Avant toute décision de trading, le backend de production screene 36+ actifs en parallèle pour identifier les opportunités les plus exploitables.

4.1. Univers d'actifs (36 tickers, 9 catégories)

Catégorie	Tickers	N
US_TECH	NVDA, AAPL, MSFT, META, AMD, TSLA	6
US_HIGHBETA	PLTR, COIN, MSTR, MARA, IONQ, HOOD, RDDT	7
US_LEVERAGED	TQQQ, SOXL, NVDL, SPXL	4
US_FINANCE	JPM, GS, V	3
US_HEALTH	LLY, MRNA	2
US_ENERGY	XOM, CVX	2
ETF	SPY, QQQ, SOXX	3
CRYPTO	BTC-USD, ETH-USD, SOL-USD, BNB-USD, XRP-USD, DOGE-USD	6
CAC40	MC.PA, TTE.PA, AIR.PA	3
Total		**36**

4.2. Architecture parallèle

Tous les tickers sont téléchargés et scorés de manière concurrente via `ThreadPoolExecutor` (12 workers, timeout 30s), maintenant le temps total de screening sous 10 secondes quelle que soit la

taille de l'univers.

```
with ThreadPoolExecutor(max_workers=12) as executor:
    futures = {
        executor.submit(_screen_single, ticker, category): ticker
        for category, tickers in UNIVERSE.items()
        for ticker in tickers
    }
    for future in as_completed(futures, timeout=30):
        result = future.result()
        if result:
            screened.append(result)
```

Listing 4 – *Screening parallèle — bot_screener.py*

4.3. Indicateurs techniques calculés

Neuf indicateurs sont calculés à partir de 6 mois de données OHLCV journalières par actif :

Indicateur	Formule / Logique	Type de signal
RSI(14)	EMA Wilder des hausses/baisses	Zone momentum
MACD(12,26,9)	$\text{EMA}_{12} - \text{EMA}_{26}$; signal $= \text{EMA}_9(\text{MACD})$	Direction tendance
Pente hist. MACD	$\Delta(\text{MACD} - \text{signal})$	Accél. momentum
Bandes de Bollinger	$\mu_{20} \pm 2\sigma_{20}$ sur les clôtures	Bande de volatilité
ADX(14)	Indice de mouvement directionnel	Force de tendance
ATR%(14)	$\frac{\text{ATR}_{14}}{P_t} \times 100$	Volatilité intraday
Volume surge	$V_t \,/\, \bar{V}_{20}$	Flux institutionnel
Momentum 1j	$(P_t - P_{t-1})/P_{t-1}$	Signal intraday
Momentum 5j	$(P_t - P_{t-5})/P_{t-5}$	Tendance hebdo

4.4. Fonction de score composite

Chaque actif reçoit un score $s_i \in [-20, +20]$ issu d'un modèle additif pondéré par morceaux :

$$s_i = \underbrace{s_{\text{RSI}}}_{[-3,5,+3]} + \underbrace{s_{\text{MACD}}}_{[-2,5,+3,5]} + \underbrace{s_{\text{ADX}}}_{[-1,5,+3,5]} + \underbrace{s_{\text{vol}}}_{[-1,+3,5]} + \underbrace{s_{\text{mom5j}}}_{[-2,5,+2,5]} + \underbrace{s_{\text{mom1j}}}_{[-1,+1,5]} + \underbrace{s_{\text{BB}}}_{[-2,+1,5]} + \underbrace{s_{\text{ATR}}}_{[-1,+1]} + \underbrace{s_{\text{cat}}}_{[0,+1]} \tag{1}$$

```
# RSI: zone momentum ideale = 50-68 (confirme, pas surachet)
if 50 <= rsi <= 68:    score += 3.0   # momentum parfait
elif 42 <= rsi < 50:   score += 1.5   # bonne entree
elif rsi < 30:         score += 2.0   # rebond survendu
elif rsi > 80:         score -= 3.5   # extremement surachet -> danger
elif rsi > 72:         score -= 1.5

# MACD: signal directionnel principal
```

```
if macd_signal == "BULL":   score += 3.5
elif macd_signal == "BEAR": score -= 2.5

# ADX: force de tendance (>25 = tendance, >40 = tres forte)
if adx >= 45:    score += 3.5    # tendance exceptionnellement forte
elif adx >= 35:  score += 2.5
elif adx >= 25:  score += 1.5
elif adx < 18:   score -= 1.5    # sans direction -> eviter

# Volume surge: confirmation institutionnelle
if volume_surge > 4.0:   score += 3.5   # explosion institutionnelle
elif volume_surge > 2.5: score += 2.0
elif volume_surge > 1.8: score += 1.0
elif volume_surge < 0.6: score -= 1.0   # volume mort -> eviter
```

Listing 5 – *Règles de scoring — _score_asset() dans bot_screener.py*

5. Moteur de décision LLM — `bot_brain.py`

Après le screening, les actifs les mieux classés sont transmis à un LLM qui synthétise l'ensemble des signaux en décisions de trading structurées. C'est la couche d'intelligence centrale du bot de production.

5.1. Cascade de modèles

Priorité	Modèle	Fournisseur	Rôle
1	gemini-2.5-flash-preview	Google	Principal
2	gemini-2.0-flash	Google	Secours 1
3	gemini-1.5-flash	Google	Secours 2
4	gemini-1.5-pro	Google	Secours 3
5	llama-3.3-70b-versatile	Groq	Dernier recours

5.2. Couche d'apprentissage ML

Le bot suit les performances par ticker sur les 30 derniers trades complétés. Ce mécanisme d'apprentissage en ligne léger alimente directement le prompt LLM sous forme de contexte ML explicite :

Tag	Condition	Effet sur les décisions LLM
STAR	Taux de gain $\geq 60\%$, ≥ 5 trades	Confiance élevée, taille augmentée
OK	$40\% \leq \text{TG} < 60\%$	Comportement standard
EVITE	$20\% \leq \text{TG} < 40\%$	Confiance réduite
BANNI	$\text{TG} < 20\%$, ≥ 5 trades	Exclu de toute décision

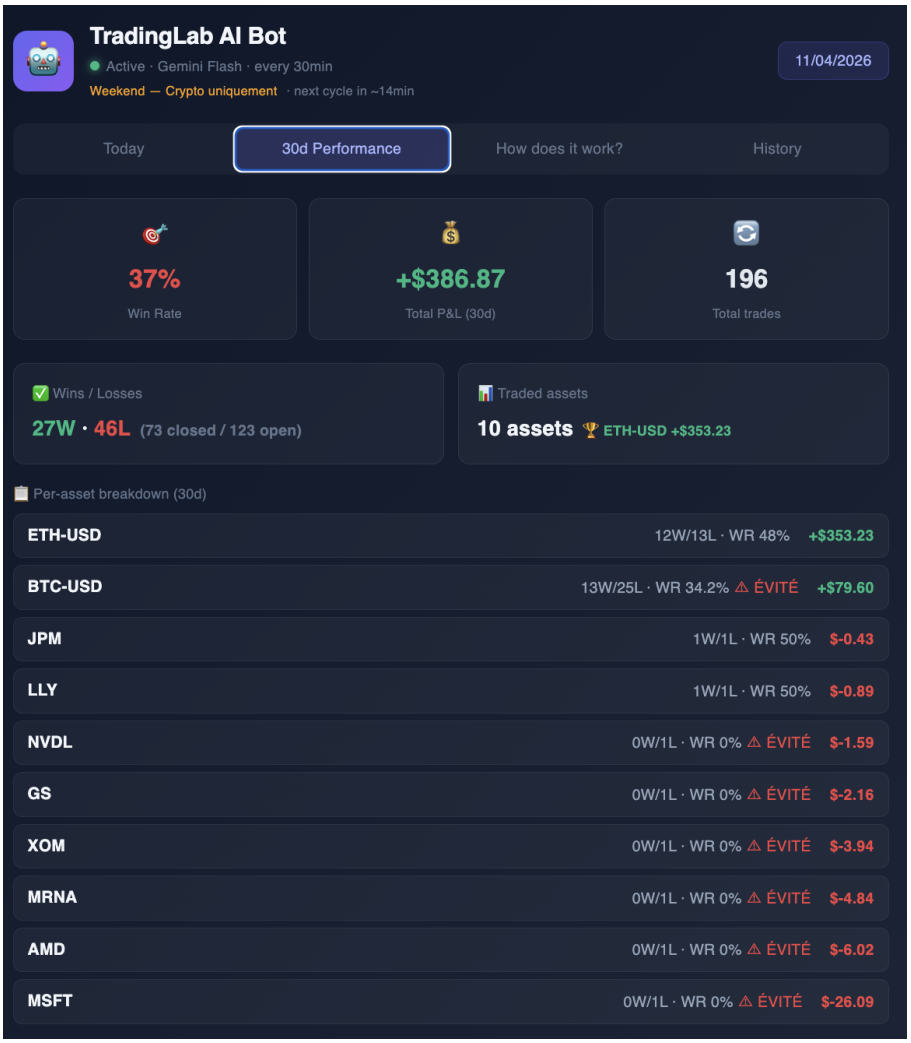


Figure 3 – *Tableau de bord de performance du bot — P&L cumulé et suivi du taux de gain par ticker (couche ML)*

5.3. Prompt engineering — Cinq blocs de contexte

Le prompt LLM est assemblé à partir de cinq blocs de contexte structurés à chaque cycle de décision :

1. **Contexte ML** — taux de gain par ticker, tags STAR/EVITE, liste des tickers bannis
2. **Contexte marché** — tendance SPY, niveau VIX, régime (NORMAL / ÉLEVÉ / EXTRÊME)
3. **Positions ouvertes** — trades en cours avec alertes TP/SL (ex. `"NVDA : +13,8% → proche du TP"`)
4. **Actifs screenés** — top 10 par score composite avec les 9 indicateurs
5. **Contraintes strictes** — règles que le modèle doit impérativement respecter (cf. §5.4)

5.4. Contraintes imposées au LLM

Règles appliquées dans bot_brain.py :
— Maximum **3 décisions par cycle** (ACHAT + VENTE combinés)
— Taille de position par confiance : **HAUTE → 20%**, MOYENNE → 13%, FAIBLE → 8% du portfolio

- Minimum **5 conditions d'entrée** requises avant tout achat
- Take-profit : **+15%** ; Stop-loss : −5% standard / −8% étendu
- Pyramidage : **+10%** sur les positions gagnantes (si conditions remplies)
- Multiplicateur régime VIX : $< 25 \rightarrow \times 1{,}0$; $25–35 \rightarrow \times 0{,}5$; $> 35 \rightarrow \times 0{,}25$
- Maximum **4 positions ouvertes simultanément**
- **Diversification** : pas deux positions de la même catégorie d'actifs

5.5. Format de sortie JSON structuré

La sortie du LLM est validée contre un schéma strict avant toute exécution :

```
{
  "decisions": [
    {
      "action":     "BUY",        // BUY | SELL | HOLD
      "ticker":     "NVDA",
      "confidence": "HIGH",       // HIGH | MEDIUM | LOW
      "amount_usd": 2000,
      "rationale_fr": "...",      // justifications bilingues
      "rationale_en": "..."
    }
  ],
  "market_commentary": "..."      // lecture globale du marche
}
```

Listing 6 – *Format de sortie attendu du LLM — bot_brain.py*

6. Actualités & Sentiment macro — `news.py`

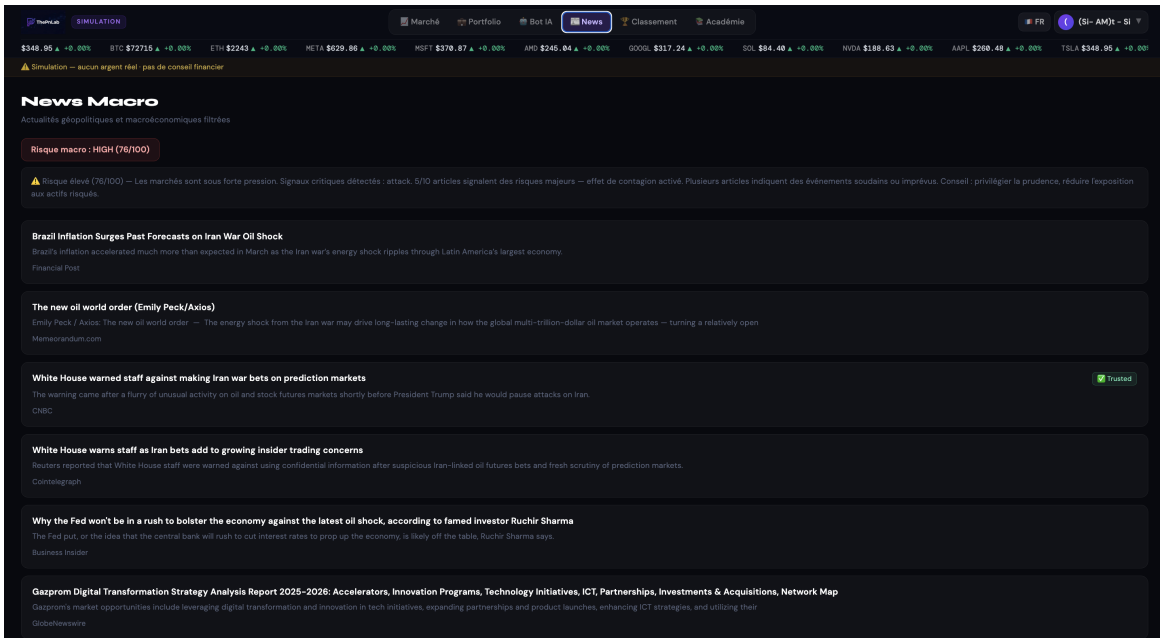


Figure 4 – *Onglet Actualités — titres macro filtrés, scorés et affichés avec évaluation du risque*

6.1. Filtrage des sources

Les titres sont récupérés depuis NewsAPI et pré-filtrés pour ne conserver que 12 sources financières de confiance :
Reuters, Bloomberg, Financial Times, Wall Street Journal, CNBC, MarketWatch, Seeking Alpha, Barron's, The Economist, Les Échos, Le Monde, Le Figaro.

Des heuristiques de détection publicitaire (liste noire de mots-clés) nettoient ensuite le flux avant le scoring.

6.2. Scoring de pertinence macro

Chaque titre est scoré sur 50+ mots-clés macro regroupés par catégorie pondérée :

Catégorie	Exemples de mots-clés	Poids
Politique monétaire	fed, hausse de taux, inflation, BCE, pivot	+3
Événements marchés	résultats, krach, rallye, IPO, squeeze	+2
Géopolitique	guerre, sanction, pétrole, OPEP, tarif	+2
Économie	PIB, chômage, récession, IPC	+1

6.3. Score de risque → multiplicateur de position

Le score de risque macro agrégé (0–100) est converti en multiplicateur de taille de position :

$$m_{\text{news}} = \begin{cases} 0{,}4 & \text{si score de risque} \geq 70 \quad \text{(risque ÉLEVÉ)} \\ 0{,}7 & \text{si } 40 \leq \text{score} < 70 \quad \text{(risque MOYEN)} \\ 1{,}0 & \text{si score} < 40 \quad \text{(risque FAIBLE)} \end{cases} \tag{2}$$

7. Gestion du risque

7.1. Pipeline de dimensionnement en 4 couches

La taille finale de position combine quatre couches de risque indépendantes de façon multiplicative :

$$\text{montant}_{\text{final}} = \underbrace{V \cdot c_{\max}}_{\text{base}} \times \underbrace{m_{\text{conf}}}_{\text{LLM}} \times \underbrace{m_{\text{VIX}}}_{\text{régime}} \times \underbrace{m_{\text{news}}}_{\text{macro}} \tag{3}$$

où V = valeur du portfolio, $c_{\max} = 15\%$, et :

Couche	Valeurs du multiplicateur	Source
m_{conf} (LLM)	HAUTE : 1,33× ; MOYENNE : 0,87× ; FAIBLE : 0,53×	`bot_brain.py`
m_{VIX} (régime)	<25 : 1,0 ; 25–35 : 0,5 ; >35 : 0,25	VIX temps réel
m_{news} (macro)	FAIBLE : 1,0 ; MOYEN : 0,7 ; ÉLEVÉ : 0,4	`news.py`

Scénario le plus conservateur : VIX > 35 + risque macro ÉLEVÉ + confiance LLM FAIBLE $\Rightarrow$ allocation effective $= 15\% \times 0{,}53 \times 0{,}25 \times 0{,}40 = \mathbf{0{,}8}\%$ du portfolio par trade.

7.2. Stratégie de sortie

Type de sortie	Déclencheur	Action
Take-profit	+15% depuis l'entrée	Vente de la position entière
Stop-loss standard	−5% depuis l'entrée	Vente de la position entière
Stop-loss étendu	−8% depuis l'entrée	Vente de la position entière
Pyramidage	+10% depuis l'entrée	Renforcement (si conditions)

8. Lien avec la Recherche Opérationnelle

ThePnLab s'inscrit directement dans trois domaines fondamentaux de la RO, directement couverts dans des cursus comme le Columbia MSOR :

8.1. Prise de décision séquentielle — Formalisation MDP

Le cycle de trading quotidien est structurellement identique à un Processus de Décision Markovien (PDM) à horizon fini :

Composante PDM	Correspondance ThePnLab
État S_t	(portfolio, scores screenés, VIX_t, taux de gain par ticker)
Action A_t	vecteur d'allocation $(a_1, \ldots, a_{36}) \in \mathbb{R}^{36}$
Transition $P(S_{t+1}\|S_t, A_t)$	dynamiques stochastiques de marché (non-stationnaires)
Récompense R_t	PnL ajusté du risque, net de frais et slippage
Politique π	actuellement : heuristique LLM $\rightarrow$ prochaine étape : agent RL

8.2. Aide multicritère à la décision (AMCD)

Le score composite du screener est un **modèle additif pondéré** — approche classique d'AMCD :

$$s_i = \sum_{k=1}^{9} w_k \cdot \phi_k(x_{ik}) \tag{4}$$

où ϕ_k est une fonction de scoring par morceaux pour l'indicateur k. Les poids sont actuellement

calibrés à la main ; une extension naturelle est l'analyse de sensibilité de Sobol ou la méthode DEA pour dériver des poids fondés sur les données.

8.3. Contrôle stochastique & changement de régime

Le contrôleur de régime VIX et le multiplicateur de risque news forment ensemble un **modèle à changements de régime**, analogue aux Modèles de Markov Cachés (MMC) :

$$\text{régime}_t \in \{\text{NORMAL, ÉLEVÉ, EXTRÊME}\} \sim \text{MMC avec VIX}_t \text{ comme variable d'émission} \quad (5)$$

9. Résultats & Validation

9.1. Performance live — 30 derniers jours

P&L total	Taux de gain	Trades	Utilisateurs
+386,87 $	**37%**	**196**	**37**

Résultats live sur 30 jours (bot de production) :
— P&L total : **+386,87 $** sur un capital initial de 10 000 $ (+3,87%)
— Taux de gain : **37%** — 27 victoires / 46 pertes (73 trades clôturés)
— Trades exécutés : **196** au total (73 clôturés + 123 positions ouvertes)
— Actifs tradés : **10 tickers actifs** sur l'univers de 36 actifs
— Meilleur actif : **ETH-USD** à **+353,23 $** (91% du P&L total)
— Plateforme opérationnelle en continu, **37 utilisateurs inscrits** — organique

Note statistique & interprétation : Un taux de gain de 37% est inférieur au seuil naïf de 50%, et pourtant le bot est profitable. Cela est cohérent avec une stratégie à *espérance positive* : les trades gagnants sont en moyenne plus importants que les perdants (ratio gain/perte $b > 1$), ce qui est précisément le régime dans lequel le dimensionnement fractional Kelly maximise la croissance à long terme. Avec 73 observations clôturées, l'IC à 95% sur le ratio de Sharpe reste large ($\approx [-1{,}5, +4]$). La contribution disproportionnée d'ETH-USD (+91% du P&L total) signale un risque de concentration — une contrainte de diversification plus stricte serait une amélioration naturelle.

10. Limitations & Feuille de route

10.1. Limitations actuelles

L1. Pas de backtesting — simulation prospective uniquement ; pas de rejeu historique ni de validation walk-forward

L2. Pas d'API de courtage — choix délibéré (réglementaire) ; limite le réalisme de l'exécution

L3. Non-déterminisme LLM — sorties Gemini stochastiques (température 0,4) ; décisions non reproductibles

L4. Screener aveugle aux corrélations — actifs scorés indépendamment ; pas de modèle de corrélation inter-actifs

L5. Impact marché non modélisé — slippage (5 bps) et frais (0,1%) implémentés, mais l'impact carnet d'ordres pour de gros ordres est absent

L6. Poids AMCD calibrés manuellement — pas d'analyse de sensibilité formelle sur les poids du score composite

10.2. Feuille de route recherche

R1. Moteur de backtesting — backtest vectorisé pandas avec validation walk-forward et modélisation des coûts de transaction

R2. Agent RL de trading — remplacer l'heuristique LLM par un agent PPO/DQN ; formalisation MDP complète (cf. §8.1)

R3. Dimensionnement par optimisation de variance — moyenne-variance : $\min_w \; w^\top \Sigma w$ s.c. $\mu^\top w \geq r_{\min}$, $\mathbf{1}^\top w = 1$

R4. Calibration HMM — ajuster un Modèle de Markov Caché sur le VIX pour remplacer les règles de régime à seuils

R5. Analyse de sensibilité de Sobol — quantifier l'incertitude des poids dans la fonction de score composite

R6. API Alpaca Paper — connexion à un courtier papier réaliste pour améliorer la modélisation du slippage

11. Références

[1] Kelly, J.L. (1956). « A New Interpretation of Information Rate. » *Bell System Technical Journal*, 35(4), 917–926. doi:10.1002/j.1538-7305.1956.tb03809.x

[2] Markowitz, H. (1952). « Portfolio Selection. » *Journal of Finance*, 7(1), 77–91. doi:10.2307/2975974

[3] Wilder, J.W. (1978). *New Concepts in Technical Trading Systems*. Trend Research. *[Formules RSI et ATR/ADX]*

[4] Appel, G. (2005). *Technical Analysis : Power Tools for Active Investors*. FT Press. *[MACD]*

[5] Sutton, R.S., Barto, A.G. (2018). *Reinforcement Learning : An Introduction* (2e éd.). MIT Press. MIT Press

[6] Merton, R.C. (1969). « Lifetime Portfolio Selection under Uncertainty. » *Review of Economics and Statistics*, 51(3), 247–257.

[7] Ledoit, O., Wolf, M. (2004). « A well-conditioned estimator for large-dimensional covariance matrices. » *Journal of Multivariate Analysis*, 88(2), 365–411.

[8] Saltelli, A. et al. (2008). *Global Sensitivity Analysis : The Primer*. Wiley. *[Indices de Sobol pour l'analyse de sensibilité AMCD]*

[9] Google DeepMind (2024). *Gemini 2.0 Flash Technical Report*. deepmind.google

[10] Groq Inc. (2024). *Llama-3.3-70B on Groq LPU inference*. groq.com

Hamza Houir Alami, ENSAM Lille, 2026
thepnlab.com h.houir.alami@gmail.com